

# Регрессия: полная теория
# House Prices — Advanced Regression Techniques

От OLS до регуляризации — полный путь ML-инженера

🎓 **ТИУ** · IT-направление

👥 Группа: **ПКТб-23-1**

📊 Датасет: **House Prices** (Kaggle)

🎯 Признаков: **79** · Строк: **1460 train + 1459 test**

📐 Формат: **A4 Landscape**

🗓️ Учебный год: **2025/2026**

---

Конспект теории · 40 слайдов · Полный пайплайн регрессии

Включает: OLS, Ridge, Lasso, ElasticNet, кросс-валидацию, GridSearchCV, диагностику

# Содержание


**Часть 1. Введение в регрессию** 3

Что такое регрессия 3

Регрессия vs Классификация 3

Датасет House Prices 4

Структура и особенности 4

**Часть 2. Линейная регрессия и OLS** 5

Модель линейной регрессии 5

Метод наименьших квадратов 5

Свойства OLS 6

**Часть 3. Регуляризация** 7

Зачем нужна регуляризация 7

Ridge (L2) 7

Lasso (L1) 8

Геометрическая интерпретация 8

ElasticNet 9

Сравнение методов 9

**Часть 4. Предобработка данных** 10

Типы пропусков 10

Стратегии обработки 10

Обработка выбросов 11

Логарифмирование целевой 11

Кодирование категорий 12

Масштабирование 12

Feature Engineering 13

**Часть 5. Метрики и валидация** 14

Метрики регрессии 14

Кросс-валидация 14

GridSearchCV 15

Утечка данных 15

Анализ остатков 16

Переобучение и недообучение 16

Важность признаков 17

**Часть 6. Практика и выводы** 18

Полный пайплайн 18

Ключевые формулы 18

Когда что использовать 19

Ожидаемые результаты 19

Частые ошибки 20

Чек-лист 20

Ключевые выводы 21

Источники 21

ЧАСТЬ 1

# Введение в регрессию

## Что такое регрессия?

**Регрессия** — это класс задач машинного обучения с учителем, в которых целевая переменная является **непрерывной величиной**. Модель учится предсказывать число, а не метку класса.

### Математическая постановка

**Дано:** обучающая выборка

$$D = \{(x_1, y_1), \ldots, (x_n, y_n)\}$$

- $\mathbf{x}_i \in \mathbb{R}^p$ — вектор из p признаков
- $\mathbf{y}_i \in \mathbb{R}$ — непрерывная целевая

**Найти:** функцию $f: \mathbb{R}^p \to \mathbb{R}$, минимизирующую ошибку предсказания.

## Регрессия vs Классификация

| Аспект | Регрессия | Классификация |
|---|---|---|
| **Целевая** | Непрерывная | Дискретная |
| **Пример** | $250 000 | 0 или 1 |
| **Выход** | Число | Вероятность |
| **Метрики** | RMSE, MAE, $R^2$ | Accuracy, F1 |
| **Потери** | MSE, MAE | Log-Loss |

## Датасет House Prices

Датасет создан **Дином Де Коком (Dean De Cock)** в 2011 году как современная замена Boston Housing Dataset.

Содержит данные о жилых домах в **Эймсе, штат Айова, США**, проданных в 2006–2010 гг.

### Задача

Предсказать **цену продажи жилья** на основе **79 признаков**.

- 🎯 Тип задачи: **регрессия**
- 📊 Целевая: `SalePrice`

| 2919 | 1460 | 1459 | 79 |
|---|---|---|---|
| ВСЕГО | TRAIN | TEST | ПРИЗНАКОВ |

### Структура признаков

| Тип | Кол-во | Примеры |
|---|---|---|
| Числовые | 36 | LotArea, YearBuilt, GrLivArea |
| Порядковые | 23 | ExterQual, BsmtQual, KitchenQual |
| Номинальные | 20 | Neighborhood, SaleType, GarageType |

> ⚠️ **Особенности**
>
> Реалистичные пропуски, скошенная целевая, мультиколлинеарность, выбросы, риск переобучения — всё, что делает датасет идеальным для обучения.

ЧАСТЬ 2

# Линейная регрессия и OLS

## Модель линейной регрессии

$$\hat{y} = w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \ldots + w_p \cdot x_p$$

где $w_0$ — смещение (bias), $w_1,\ldots,w_p$ — веса (коэффициенты).

### В матричной форме

$$\hat{y} = X \cdot w$$

- $X \in \mathbb{R}^{n \times (p+1)}$ — матрица признаков
- $w \in \mathbb{R}^{p+1}$ — вектор весов
- $n$ — число объектов, $p$ — число признаков

### Идея метода

Модель предполагает **линейную зависимость** целевой от признаков. Каждый признак влияет на целевую **независимо** с весом $w_j$.

## Метод наименьших квадратов (OLS)

**OLS (Ordinary Least Squares)** — находит веса, при которых сумма квадратов отклонений предсказаний от истинных значений минимальна.

### Функция потерь (MSE)

$$L(w) = (1/n) \cdot \Sigma_i \, (y_i - \hat{y}_i)^2$$

### Аналитическое решение

Чтобы найти минимум, берём производную по $w$ и приравниваем к нулю:

$$\hat{w} = (X^T X)^{-1} \cdot X^T \cdot y$$

Это **нормальное уравнение**. Работает, если $X^T X$ обратима.

### ⚠️ Проблемы OLS

- **Переобучение** при $p \approx n$ или $p > n$
- **Мультиколлинеарность** → $X^T X$ плохо обусловлена
- **Выбросы** сильно искажают веса
- **Скошенность** целевой нарушает предпосылки

## Свойства OLS-оценки

### 🎯 Несмещённость

$E[\hat{w}] = w_{true}$ при выполнении предпосылок.

### ⚡ Эффективность

Наименьшая дисперсия (теорема Гаусса-Маркова).

### 📈 Состоятельность

$\hat{w} \to w_{true}$ при $n \to \infty$.

### 📋 Предпосылки

Линейность, независимость, $E[\varepsilon|x]=0$, гомоскедастичность, нет мультиколлинеарности.

ЧАСТЬ 3

# Регуляризация: Ridge, Lasso, ElasticNet

## Зачем нужна регуляризация?

🚨 При **большом числе признаков** (в House Prices — 79) OLS-решение **переобучается**: модель запоминает шум в обучающих данных, а на новых данных показывает плохой результат.

⚠️ **Симптомы переобучения**

- Ошибка на train очень низкая
- Ошибка на test высокая
- Веса модели огромные по модулю
- Модель чувствительна к малейшим изменениям

✅ **Идея регуляризации**

Добавить **штраф за большие веса** к функции потерь.

$$L = MSE + \alpha \cdot Штраф(w)$$

Это заставляет модель искать **простое** решение.

## Ridge-регрессия (L2)

$$L_{Ridge}(w) = MSE + \alpha \cdot \Sigma_j \, w_j^2$$

**Аналитическое решение:**

$$\hat{w} = (X^T X + \alpha I)^{-1} \cdot X^T \cdot y$$

Добавление **αI** гарантирует обратимость матрицы **$X^TX$** даже при мультиколлинеарности.

- Веса **уменьшаются**, но **не обнуляются**
- Хорошо работает при мультиколлинеарности
- Не выполняет отбор признаков
- Эквивалентна MAP-оценке с гауссовским априорным распределением

## Lasso-регрессия (L1)

$$L_{Lasso}(w) = MSE + \alpha \cdot \Sigma_j \, |w_j|$$

L1-штраф имеет **«углы»** на осях. При оптимизации минимум часто попадает **именно в угол** — некоторые веса **точно обнуляются**.

- **Автоматический отбор признаков**
- Разреженное решение (sparse)
- Устойчивость к выбросам
- Эквивалентна MAP с лапласовским априорным

⚠️ При сильно коррелированных признаках Lasso выбирает **произвольный** — может оставить «не тот».

# Регуляризация (продолжение)

## Геометрическая интерпретация

Регуляризация — это **ограничение** на область допустимых весов. Форма ограничения определяет поведение:

### ⭕ Ridge (L2)

**Ограничение:** $||w||_2^2 \leq t$ — **круг** (сфера в p-мерном пространстве).

Минимум MSE + штраф обычно **касается границы круга** — обе координаты ненулевые.

➡️ Веса уменьшаются, но не обнуляются.

### 💎 Lasso (L1)

**Ограничение:** $||w||_1 \leq t$ — **ромб**.

У ромба есть **вершины на осях**. Минимум часто попадает **в угол** — одна координата обнуляется.

➡️ Автоматический отбор признаков!

🎯 Именно **форма ограничения** (круг vs ромб) объясняет разницу в поведении Ridge и Lasso.

## ElasticNet — компромисс

$$L(w) = MSE + \alpha \cdot (\rho \cdot \Sigma|w_j| + (1-\rho)/2 \cdot \Sigma w_j^2)$$

где **α** — общая сила регуляризации, **ρ ∈ [0, 1]** — баланс L1/L2.

- **ρ = 0** → чистый Ridge
- **ρ = 1** → чистый Lasso
- **0 < ρ < 1** → компромисс

### Преимущества

- Работает лучше Lasso при коррелированных признаках
- Работает лучше Ridge при большом числе признаков
- Стабильнее Lasso
- Может выбрать группу коррелированных признаков

## Сравнение методов

| Метод | Штраф | Отбор | Мультикол. | Гиперпар. |
|---|---|---|---|---|
| **OLS** | — | ❌ | ❌ | — |
| **Ridge** | L2 | ❌ | ✅ | α |
| **Lasso** | L1 | ✅ | ⚠️ | α |
| **ElasticNet** | L1+L2 | ✅ | ✅ | α, ρ |

📊 **Ridge**

Когда **все признаки полезны**, но есть мультиколлинеарность.

🎯 **Lasso**

Когда нужно **уменьшить число признаков**.

🔀 **ElasticNet**

Когда **много коррелированных** признаков — золотая середина.

💡 **Выбор α:** чем больше α — тем сильнее штраф. При α → 0 получаем OLS. При α → ∞ все веса → 0.

ЧАСТЬ 4

# Предобработка данных

## Типы пропусков по механизму

| Тип | Аббревиатура | Описание | Пример в House Prices |
|---|---|---|---|
| **Полностью случайные** | MCAR | Пропуск не зависит ни от чего | Редкий случай |
| **Случайные** | MAR | Пропуск зависит от других признаков | LotFrontage зависит от Neighborhood |
| **Неслучайные** | MNAR | Пропуск зависит от самого значения | PoolQC = NaN, потому что нет бассейна |

🎯 **Ключевая идея:** стратегия обработки зависит от **типа пропуска**, а не только от процента!

## Стратегии обработки пропусков

| Стратегия | Когда применять | Плюсы | Минусы |
|---|---|---|---|
| 🗑️ Удаление строк | < 5% пропусков | Простота | Потеря данных |
| ❌ Удаление столбца | > 50% пропусков | Простота | Потеря признака |
| 📊 Среднее | Нормальное распр. | Простота | Искажает распределение |
| 📈 Медиана | Числовые, выбросы | Устойчива к выбросам | Не учитывает группы |
| 🎯 Мода | Категориальные | Простота | Перекос |
| 👥 Групповая медиана | Связь с признаками | Точнее | Сложнее |
| 🤖 KNN-импутация | Сложные зависимости | Учитывает похожих | Дорого |
| 🆕 "None" / 0 | Смысловые пропуски | Сохраняет информацию | Только для MNAR |

💡 **Для House Prices:** 19 столбцов с пропусками → используем **3 стратегии** в зависимости от типа.

## Обработка выбросов

### 🔢 Метод IQR

$$IQR = Q_3 - Q_1$$

**Границы:**

- Нижняя: $Q_1 - 1.5 \cdot IQR$
- Верхняя: $Q_3 + 1.5 \cdot IQR$

Не требует нормальности, устойчив к выбросам.

### 📊 Z-score

$$z = (x - \mu) / \sigma$$

**Правило:** $|z| > 3 \rightarrow$ выброс.

Предполагает нормальность. Плохо работает на скошенных данных.

### 🔧 Модифицированный Z-score

$$M = 0.6745 \cdot (x - median) / MAD$$

Использует медиану и MAD вместо среднего и std. Устойчив к выбросам.

### 🎯 В House Prices

Известные выбросы по документации Kaggle:

- GrLivArea > 4000 кв. футов
- SalePrice < $300 000

Удаляем только из **train**!

# Предобработка (продолжение)

## Логарифмирование скошенной целевой

⚠️ **Проблема скошенности**

`SalePrice` имеет **правую асимметрию** (skew ≈ 1.88):

- Большинство домов дешёвые
- Несколько домов очень дорогие
- Распределение далеко от нормального

Последствия: гетероскедастичность, большое влияние выбросов.

✅ **Решение: log1p**

```
y' = log(1 + y)
```

**Обратное:**

```
y = exp(y') - 1
```

skew после: ≈ 0.12 — почти нормальное!

⚠️ **Важно:** после логарифмирования метрики (RMSE) считаются **в лог-пространстве**. Для реального RMSE в долларах используйте `np.expm1(predictions)`.

## Кодирование категориальных признаков

| Метод | Когда применять | Плюсы | Минусы |
|---|---|---|---|
| **Label Encoding** | Порядковые (ExterQual, BsmtQual) | Простота, сохраняет порядок | Ложный порядок для номинальных |
| **One-Hot Encoding** | Номинальные с малым числом категорий (MSZoning) | Нет ложного порядка | Взрыв размерности |
| **Target Encoding** | Номинальные с 20+ категорий (Neighborhood) | Компактность, учитывает целевую | Риск утечки данных |

## Масштабирование признаков

### 📊 StandardScaler

$$z = (x - \mu) / \sigma$$

mean = 0, std = 1

Для LR, Ridge, Lasso, SVM.

### 📐 MinMaxScaler

$$z = (x - min) / (max - min)$$

Диапазон [0, 1]

Для нейросетей.

### 💪 RobustScaler

$$z = (x - median) / IQR$$

Устойчив к выбросам

Для «грязных» данных.

### 🚨 Критично

`fit_transform()` только на **train**! На test — только `transform()`. Иначе — **утечка данных**.

## Feature Engineering

### 🔢 Агрегации

- **TotalSF** = TotalBsmtSF + 1stFlrSF + 2ndFlrSF
- **TotalBath** = FullBath + 0.5·HalfBath + BsmtFullBath + 0.5·BsmtHalfBath
- **TotalPorch** = OpenPorchSF + EnclosedPorch + 3SsnPorch + ScreenPorch + WoodDeckSF

### 🗓️ Возрастные

- **HouseAge** = YrSold – YearBuilt
- **RemodAge** = YrSold – YearRemodAdd
- **IsRemodeled** = 1 если была реконструкция

### 🎯 Бинарные

- **HasPool** = PoolArea > 0
- **HasGarage** = GarageArea > 0
- **Has2ndFloor** = 2ndFlrSF > 0
- **HasFireplace** = Fireplaces > 0

### 🔀 Взаимодействия

- **OverallQual_TotalSF** = OverallQual × TotalSF
- **Qual_Age** = OverallQual / (HouseAge + 1)
- **OverallScore** = композитный индекс качества

ЧАСТЬ 5

# Метрики и валидация

## Метрики регрессии

### 📉 MSE — Mean Squared Error

$$MSE = (1/n)\cdot\Sigma(y_i - \hat{y}_i)^2$$

Штрафует большие ошибки сильнее. Чувствителен к выбросам.

### 📈 RMSE

$$RMSE = \sqrt{MSE}$$

В единицах целевой (долларах). Чувствителен к выбросам.

### 📊 MAE

$$MAE = (1/n)\cdot\Sigma|y_i - \hat{y}_i|$$

Устойчива к выбросам. Соответствует медианной ошибке.

### 🎯 $R^2$

$$R^2 = 1 - SS_{res} / SS_{tot}$$

Доля объяснённой дисперсии. 0..1 (может быть < 0).

## Кросс-валидация (K-Fold)

### ⚠️ Проблема одного split

- Оценка зависит от случайности
- Маленький датасет → нестабильные метрики
- Нет информации о разбросе качества

### ✅ Решение: K-Fold

1. Разбить данные на K фолдов
2. Обучить K раз (каждый фолд побывает в test)
3. Усреднить метрики

| | | | |
|---|---|---|---|
| **5** | **10** | **LOO** | **CV** |
| СТАНДАРТ | ДЛЯ БОЛЬШИХ ДАННЫХ | LEAVE-ONE-OUT | УСРЕДНЕНИЕ |

```python
from sklearn.model_selection import cross_val_score, KFold

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(ridge, X_train_scaled, y_train,
                         cv=kf, scoring='neg_root_mean_squared_error')

print(f"RMSE по фолдам: {-scores}")
print(f"Среднее: {-scores.mean():.4f} ± {scores.std():.4f}")
```

## Подбор гиперпараметров (GridSearchCV)

### 📋 GridSearchCV

Перебор **всех комбинаций** из сетки параметров.

➕ Надёжно · ➖ Медленно

### 🎲 RandomizedSearchCV

Случайные N комбинаций из распределений.

➕ Быстро · ➖ Может пропустить оптимум

```python
from sklearn.model_selection import GridSearchCV

# Сетка значений alpha для Ridge
ridge_params = {'alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]}

ridge_grid = GridSearchCV(
    Ridge(random_state=42),
    ridge_params,
    cv=kf,
    scoring='neg_root_mean_squared_error',
    n_jobs=-1  # все CPU
)
ridge_grid.fit(X_train_scaled, y_train)

print(f"Best alpha: {ridge_grid.best_params_['alpha']}")
print(f"Best RMSE: {-ridge_grid.best_score_:.4f}")
```

# Диагностика и интерпретация

## Утечка данных (Data Leakage)

🚨 **Утечка данных** — ситуация, когда информация из test «просачивается» в train через preprocessing. Модель показывает **нереально высокие метрики**, но в продакшене «падает».

❌ **Типичные ошибки**

- `scaler.fit_transform(X_all)` до split
- Вычисление медианы на всём датасете
- Target Encoding до split
- Feature selection по всем данным
- Удаление выбросов из test

✅ **Правильно**

- Сначала `train_test_split`
- `fit` только на train
- `transform` на test с train-параметрами
- Использовать `Pipeline`
- Target Encoding внутри CV

```
# ❌ НЕПРАВИЛЬНО — утечка!
scaler = StandardScaler()
X_all_scaled = scaler.fit_transform(X_all)  # fit на всех данных!
X_train, X_test, y_train, y_test = train_test_split(X_all_scaled, y)

# ✅ ПРАВИЛЬНО
X_train, X_test, y_train, y_test = train_test_split(X, y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)  # только train!
X_test_scaled = scaler.transform(X_test)         # transform на test
```

## Анализ остатков

**Остатки:** $e_i = y_i - \hat{y}_i$. Их анализ показывает, **адекватна ли модель**.

### 📊 Что проверять

1. **Среднее ≈ 0** — нет систематической ошибки
2. **Нормальность** — QQ-plot
3. **Гомоскедастичность** — разброс постоянный
4. **Отсутствие автокорреляции** — Durbin-Watson

### 🎭 Паттерны остатков

| Паттерн | Проблема |
|---|---|
| Случайный разброс | ✅ Хорошо |
| Веер | Гетероскедастичность |
| U-образный | Нелинейность |
| Тренд | Смещение |

## Переобучение и недообучение

### 📉 Underfitting

**Train error: высокий**
**Test error: высокий**

Модель слишком простая.

Решение: усложнить модель, больше признаков, меньше регуляризации.

### ✅ Optimum

**Train error: низкий**
**Test error: низкий**

Модель обобщает.

Кросс-валидация даёт стабильные метрики.

### 📈 Overfitting

**Train error: очень низкий**
**Test error: высокий**

Модель запомнила шум.

Решение: регуляризация, больше данных, упрощение.

## Важность признаков

### 📊 Коэффициенты линейных моделей

`model.coef_[0]` — веса признаков.

- Положительный → повышает предсказание
- Отрицательный → понижает

⚠️ Работает только после масштабирования!

### 🎯 Lasso обнуляет

Lasso **автоматически** удаляет неинформативные признаки.

Если Lasso обнулил N признаков — они **не влияют** на целевую.

➡️ Это **встроенный отбор признаков**.

```
# Коэффициенты Ridge
coef_df = pd.DataFrame({
    'Feature': feature_cols,
    'Coefficient': ridge_model.coef_
}).sort_values('Coefficient', ascending=False)

# Визуализация ТОП-30
top_features = pd.concat([coef_df.head(15), coef_df.tail(15)])
colors = ['#2ecc71' if c > 0 else '#e74c3c' for c in top_features['Coefficient']]
plt.barh(top_features['Feature'], top_features['Coefficient'], color=colors)
plt.show()

# Сколько признаков обнулил Lasso?
n_zero = (lasso_model.coef_ == 0).sum()
print(f"Lasso обнулил: {n_zero} из {len(lasso_model.coef_)}")
```

ЧАСТЬ 6

# Практика и выводы

## Полный пайплайн регрессии

1. **Загрузка данных:** train.csv (1460×81), test.csv (1459×80)
2. **EDA:** распределения, корреляции, пропуски, выбросы
3. **Обработка пропусков:** смысловые → "None"/0, случайные → медиана по группам
4. **Обработка выбросов:** IQR + доменные знания (GrLivArea < 4000)
5. **Логарифмирование:** SalePrice_log = np.log1p(SalePrice)
6. **Feature Engineering:** TotalSF, TotalBath, HouseAge, HasPool
7. **Кодирование:** порядковые → Label, номинальные → One-Hot
8. **Разделение:** train_test_split (80/20)
9. **Масштабирование:** StandardScaler (fit на train)
10. **Обучение:** LR, Ridge, Lasso, ElasticNet
11. **CV + GridSearchCV:** подбор $\alpha$ и $\rho$
12. **Оценка на test:** RMSE, MAE, $R^2$ (в log и в долларах)
13. **Интерпретация:** коэффициенты, важность признаков
14. **Сохранение:** joblib для модели, JSON для метаданных

# Ключевые формулы

## 📈 Модели

$$\hat{y} = w_0 + w_1 x_1 + \ldots + w_p x_p$$

$$L_{Ridge} = MSE + \alpha \cdot \Sigma w_j^2$$

$$L_{Lasso} = MSE + \alpha \cdot \Sigma |w_j|$$

## 📊 Метрики

$$MSE = (1/n) \cdot \Sigma (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$R^2 = 1 - SS_{res}/SS_{tot}$$

## 🛠️ Предобработка

$$z = (x - \mu)/\sigma$$

$$y' = \log(1 + y)$$

$$IQR = Q_3 - Q_1$$

## 🔄 Кросс-валидация

$$CV_K = (1/K) \cdot \Sigma \ Metric_i$$

$$\sigma = \sqrt{[(1/(K-1)) \cdot \Sigma (Metric_i - CV_K)^2]}$$

## Когда какую модель использовать

### 📊 Ridge

- Все признаки **полезны**
- Есть **мультиколлинеарность**
- Нужна **стабильность**

### 🎯 Lasso

- Нужен **отбор признаков**
- Много «шумных» признаков
- Хотим **интерпретируемость**

### 🔀 ElasticNet

- Признаки **коррелированы**
- Хотим лучшее из L1 и L2
- Lasso нестабилен

### 📉 OLS

- Признаков **мало** ($p << n$)
- Нет мультиколлинеарности
- Данные «чистые»

# Результаты и ошибки

## Ожидаемые результаты на House Prices

| Модель | RMSE (log) | MAE (log) | $R^2$ | RMSE ($) |
|---|---|---|---|---|
| Linear Regression (OLS) | ~0.135 | ~0.095 | ~0.88 | ~$28 000 |
| Ridge (α≈1) | ~0.128 | ~0.090 | ~0.90 | ~$26 000 |
| Lasso (α≈0.0005) | ~0.125 | ~0.088 | ~0.91 | ~$25 000 |
| ElasticNet | ~0.126 | ~0.089 | ~0.91 | ~$25 500 |

✅ **Что важно**

- **Ridge/Lasso/ElasticNet** обгоняют OLS
- Разница 5–10% по RMSE
- Lasso выбирает ~50 признаков из 200+
- Все обучаются **за секунды**

🎯 **Что улучшит результат**

- **XGBoost/LightGBM** — +3-5% к $R^2$
- **Stacking** — ансамбль моделей
- **Optuna** для гиперпараметров
- **RFE** для отбора признаков

## Частые ошибки в регрессии

1. ❌ **fit_transform() на всём датасете до split** → утечка данных
2. ❌ **Обучение на скошенной целевой без log1p** → плохой $R^2$
3. ❌ **Игнор мультиколлинеарности** → нестабильные веса
4. ❌ **Удаление выбросов из test** → утечка + потеря данных
5. ❌ **RMSE в log-пространстве как «реальный» RMSE**
6. ❌ **Сравнение моделей по метрике на train** → переобученная «победит»
7. ❌ **Игнор остатков** → не замечаем проблему модели
8. ❌ **Забыли про стратификацию распределения целевой**

# Чек-лист регрессионного проекта

### 📋 Подготовка данных

- ☑ Проверил распределение целевой (skew?)
- ☑ Обработал пропуски по типам
- ☑ Удалил выбросы из train (IQR/домен)
- ☑ Логарифмировал скошенную целевую
- ☑ Создал новые признаки
- ☑ Закодировал категории правильно

### 🎓 Моделирование

- ☑ Разделил train/test ДО масштабирования
- ☑ fit только на train
- ☑ Обучил baseline (OLS)
- ☑ Сравнил Ridge/Lasso/ElasticNet
- ☑ Провёл кросс-валидацию
- ☑ Подобрал гиперпараметры

### 📊 Оценка

- ☑ Посчитал RMSE, MAE, $R^2$
- ☑ Проанализировал остатки
- ☑ Построил график предсказаний
- ☑ Сравнил модель с baseline

### 📦 Деплой

- ☑ Интерпретировал коэффициенты
- ☑ Создал функцию предсказания
- ☑ Сохранил модель (joblib)
- ☑ Сохранил метаданные (JSON)
- ☑ Проверил загрузку из репозитория

# Ключевые выводы и источники

## Топ-10 правил регрессии

1. **Регрессия** — предсказание непрерывной величины. **Классификация** — дискретной.
2. **OLS** минимизирует MSE. Решение: $\hat{w} = (X^TX)^{-1}X^Ty$.
3. **Ridge (L2)** уменьшает веса, **Lasso (L1)** обнуляет их — отбор признаков.
4. **ElasticNet** — компромисс между Ridge и Lasso.
5. **Логарифмирование** скошенной целевой улучшает нормальность и гомоскедастичность.
6. **Пропуски** бывают смысловыми (→"None") и случайными (→ медиана).
7. **Выбросы** удаляются методом IQR или Z-score. Только из train!
8. **Кросс-валидация** даёт устойчивую оценку. **GridSearchCV** — оптимальные гиперпараметры.
9. **fit_transform** только на train. **transform** на test.
10. **$R^2$** — доля объяснённой дисперсии. **RMSE** — средняя ошибка в единицах целевой.

## Что дальше

### 📘 Модуль 2

- 🔷 SVM для классификации (Heart Disease)
- 📊 Сравнение LR и SVM
- 🎯 Метрики классификации
- ⚖️ Дисбаланс классов

### 📗 Модуль 3 — Ансамбли

- 🌳 Random Forest
- 🚀 XGBoost, LightGBM, CatBoost
- 🎛️ Optuna для гиперпараметров
- 🔍 SHAP для интерпретации

### 📙 Модуль 4 — Признаки

- 🗺️ Геоданные
- 📝 Тексты (TF-IDF)
- 🗓️ Временные ряды как признаки

### 📕 Модуль 5 — Временные ряды

- 📉 STL-декомпозиция
- 🔄 SARIMA, ARIMAX
- 📈 Multi-step forecasting

## Источники


1. Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.
2. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.
3. Kaggle. House Prices — Advanced Regression Techniques. https://www.kaggle.com/c/house-prices-advanced-regression-techniques
4. Scikit-learn Documentation. https://scikit-learn.org/stable/
5. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer.
6. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.). Springer.